

# Dokumentation Fitguide WS 25/26

---

**Projektname:** Fitguide **Team:** Christoph Fröhlich

**Datum:** 06.02.2026

---

## Inhaltsverzeichnis

- [1. Beschreibung der App](#)
  - [2. User Stories / Anforderungen](#)
  - [3. Wireframes / Mockups](#)
  - [4. Zeitplan](#)
  - [5. Installationsanweisung / Kurzanleitung](#)
  - [6. Aufbau der Anwendung](#)
  - [7. Angewendete Methoden und Bibliotheken](#)
  - [8. Technische Punkte](#)
  - [9. Tests](#)
  - [10. Aktueller Stand](#)
- 

## 1. Beschreibung der App

### App Idee

Die App richtet sich an Menschen, die ihre Gesundheit und ihr Leben verbessern möchten, aber noch unsicher sind, wie sie beim Thema Sport am besten starten sollen. Sie bietet einen schnellen, unkomplizierten Einstieg in ein gesünderes Leben durch personalisierte Trainings- und Ernährungspläne.

### Produktvision

Die Vision ist es, eine App zu entwickeln, die nach einem kurzen Onboarding direkt individualisierte Pläne für Training und Ernährung erstellt. Die App soll als erster Pfeiler der Selbstverbesserung dienen und den Nutzern eine klare Richtung weisen, ohne Überforderung, mit direktem Fokus auf das Wesentliche.

### Marktlücke

Während der Markt viele Fitness-Apps bietet, sind diese häufig überladen, kompliziert und erfordern umfangreiche Einrichtung. Die App schließt die Lücke für Einsteiger, die keine Zeit oder Lust auf stundenlange Konfiguration haben. Durch schnelles Onboarding und ein minimalistisches, intuitives Menü führt sie Nutzer direkt zum Training, ohne unnötigen Setup-Aufwand.

---

## 2. User Stories / Anforderungen

### US-01: Benutzerregistrierung

**Priorität:** Hoch

**Beschreibung:** Als neuer Benutzer möchte ich mich mit E-Mail und Passwort registrieren können, damit ich ein Konto erstellen und die App nutzen kann.

**Abhängigkeiten:** Keine

**Akzeptanzkriterien:**

- ☐ Registrierungsformular mit E-Mail und Passwort funktioniert und validiert Eingaben
  - ☐ Bei erfolgreicher Registrierung wird der Benutzer automatisch eingeloggt
  - ☐ Es werden Hinweise bei fehlerhaften Eingaben angezeigt
- 

## US-02: Onboarding-Prozess

**Priorität:** Hoch

**Beschreibung:** Als neuer Benutzer möchte ich meine persönlichen Daten in einem schrittweisen Prozess eingeben, damit die App meine Makronährstoffe automatisch berechnen und personalisierte Einstellungen vornehmen kann.

**Abhängigkeiten:** US-01 (Benutzer muss registriert sein)

**Akzeptanzkriterien:**

- ☐ 5-Schritte Onboarding erfasst Name, Körperdaten, Trainingsziel, Aktivitätslevel und Trainingshäufigkeit
  - ☐ Kalorien und Makros werden basierend auf den Eingaben automatisch berechnet
  - ☐ Nach Abschluss werden alle Daten in Firestore gespeichert
- 

## US-03: Dashboard-Übersicht

**Priorität:** Mittel

**Beschreibung:** Als Benutzer möchte ich auf dem Dashboard eine Übersicht meiner wichtigsten Fitness-Kennzahlen sehen, damit ich meinen Fortschritt auf einen Blick verfolgen kann.

**Abhängigkeiten:** US-02 (Onboarding), US-04 (Tagebuch für Daten)

**Akzeptanzkriterien:**

- ☐ Dashboard zeigt Streak, aktuelles Gewicht und Gewichtsverlauf als Diagramm
  - ☐ Makronährstoff-Übersicht zeigt tägliche Ziele an
  - ☐ Alle Daten werden automatisch aus Tagebucheinträgen aktualisiert
- 

## US-04: Tagebuch führen

**Priorität:** Mittel

**Beschreibung:** Als Benutzer möchte ich täglich meine Ernährungs- und Gewichtsdaten erfassen, damit ich meine Fortschritte dokumentieren und nachverfolgen kann.

**Abhängigkeiten:** US-02 (Onboarding für Zielwerte)

**Akzeptanzkriterien:**

- ☐ Gewicht, Kalorien und Protein können täglich eingetragen werden
  - ☐ Optional kann ein Foto und eine Notiz hochgeladen werden
  - ☐ Einträge werden in Firestore gespeichert und können bearbeitet werden
- 

## US-05: Workout erstellen

**Priorität:** Hoch

**Beschreibung:** Als Benutzer möchte ich eigene Trainingspläne mit mehreren Workouts erstellen, damit ich mein Training strukturiert planen kann.

**Abhängigkeiten:** US-06 (Übungspool muss existieren)

**Akzeptanzkriterien:**

- ☐ Übungen aus dem Pool können zu Workouts hinzugefügt werden
  - ☐ Pläne können gespeichert, bearbeitet und gelöscht werden
- 

## US-06: Übungspool verwalten

**Priorität:** Hoch

**Beschreibung:** Als Benutzer möchte ich eigene Übungen erstellen und organisieren, damit ich sie in meinen Trainingsplänen verwenden kann.

**Abhängigkeiten:** Keine

**Akzeptanzkriterien:**

- ☐ Übungen mit Name und Muskelgruppe können erstellt werden
  - ☐ Übungen können nach Muskelgruppe gefiltert werden
  - ☐ Übungen können bearbeitet und gelöscht werden
- 

## US-07: Training durchführen

**Priorität:** Hoch

**Beschreibung:** Als Benutzer möchte ich ein Workout aus meinem Trainingsplan starten und Sets loggen können, damit ich mein Training dokumentieren kann.

**Abhängigkeiten:** US-05 (Trainingsplan muss existieren) US-06 (Übungspool muss existieren)

**Akzeptanzkriterien:**

- ☐ Workout kann gestartet werden und zeigt alle Übungen korrekt an
- ☐ Für jeden Set können Gewicht und Wiederholungen eingetragen werden

- ☐ Abgeschlossenes Training wird in der Historie gespeichert
- 

## US-08: Trainingshistorie einsehen

**Priorität:** Mittel

**Beschreibung:** Als Benutzer möchte ich meine vergangenen Trainings mit allen Details ansehen, damit ich meine Entwicklung und Leistungssteigerung nachvollziehen kann.

**Abhängigkeiten:** US-07 (Trainings müssen durchgeführt worden sein)

**Akzeptanzkriterien:**

- ☐ Liste aller abgeschlossenen Trainings wird nach Datum sortiert angezeigt
  - ☐ Trainings können aufgeklappt werden, um Details zu sehen
  - ☐ Nur abgeschlossene Sets werden angezeigt
- 

## US-09: Einstellungen anpassen

**Priorität:** Mittel

**Beschreibung:** Als Benutzer möchte ich meine Makronährstoffe und Trainingsziele manuell ändern können, damit ich die App an veränderte Bedürfnisse anpassen kann.

**Abhängigkeiten:** US-02 (Onboarding)

**Akzeptanzkriterien:**

- ☐ Makronährstoffe (Kalorien, Protein, Kohlenhydrate, Fette) können manuell angepasst werden
  - ☐ Trainingsziel kann geändert werden
  - ☐ Änderungen werden gespeichert und sofort angewendet
- 

## US-10: Theme wechseln

**Priorität:** Niedrig

**Beschreibung:** Als Benutzer möchte ich zwischen Dark Mode und Light Mode wechseln können, damit ich die App nach meinen visuellen Präferenzen nutzen kann.

**Abhängigkeiten:** Keine

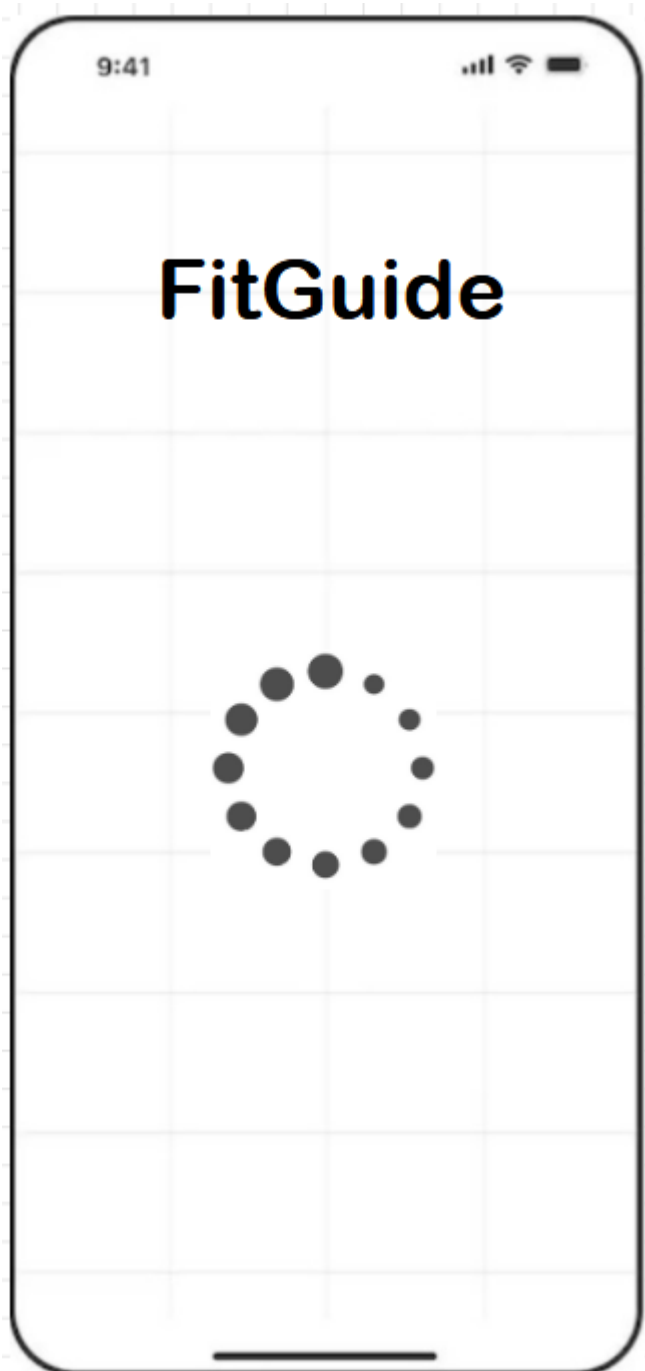
**Akzeptanzkriterien:**

- ☐ Theme-Toggle ist in den Einstellungen verfügbar
- ☐ Theme-Wechsel ist sofort sichtbar ohne App-Neustart
- ☐ Theme-Präferenz wird persistent gespeichert

## 3. Wireframes / Mockups

Splashscreen

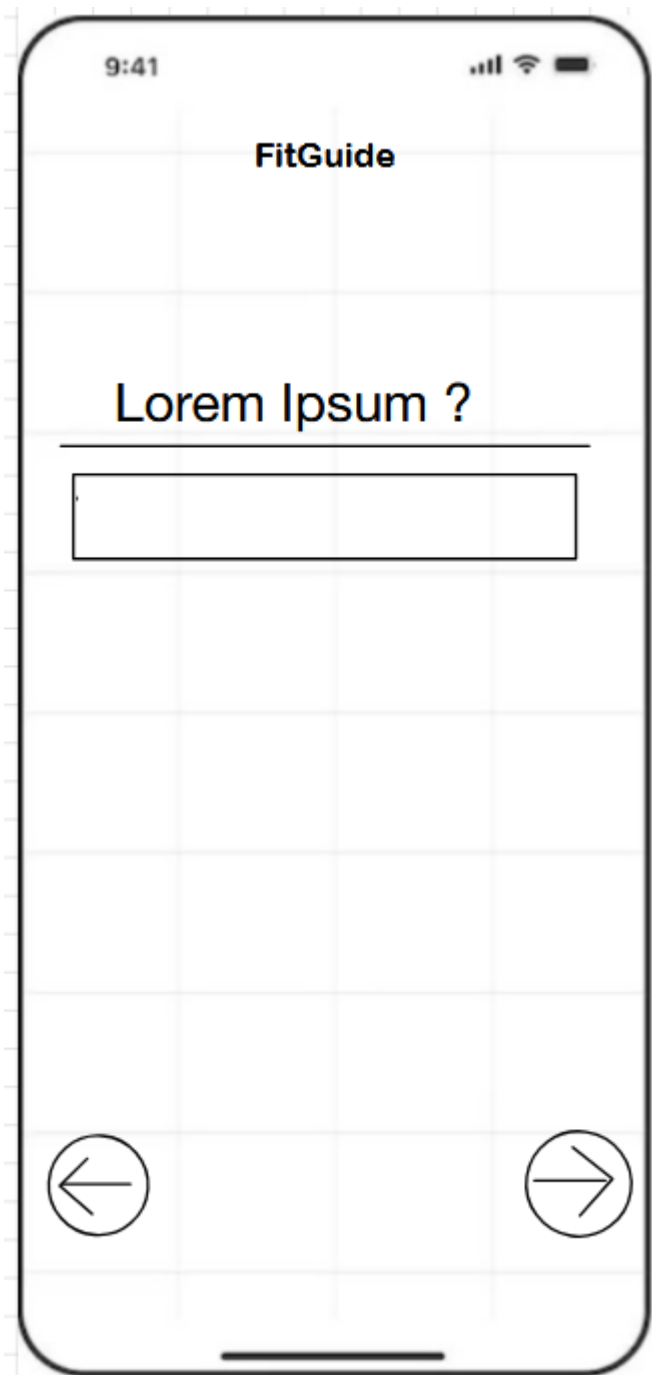
Der Startbildschirm, der beim Öffnen der App angezeigt wird.



---

## Onboarding

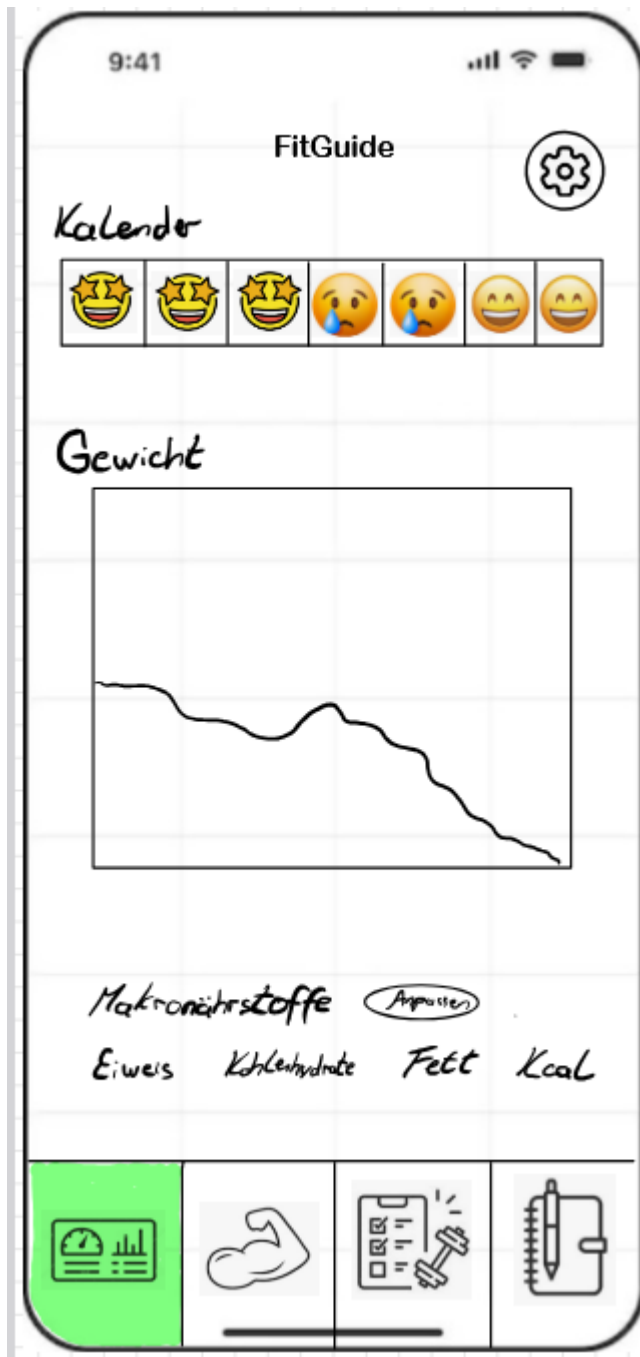
Der schrittweise Onboarding-Prozess zur Erfassung der persönlichen Daten.



---

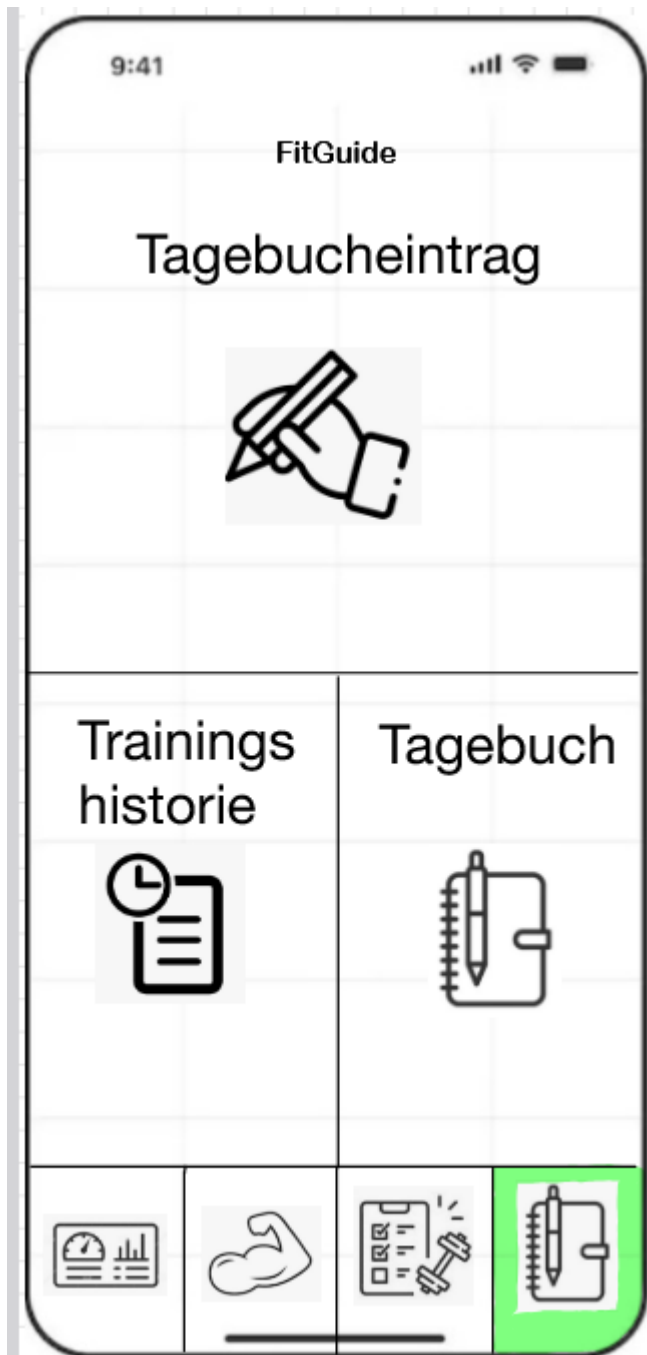
## Dashboard

Die Hauptansicht mit Streak, Gewichtsverlauf und Makronährstoff-Übersicht.



## Logbook (Tagebuch)

Tägliche Erfassung von Gewicht, Kalorien, Protein und optionalen Fotos.



## Trainingsplaner

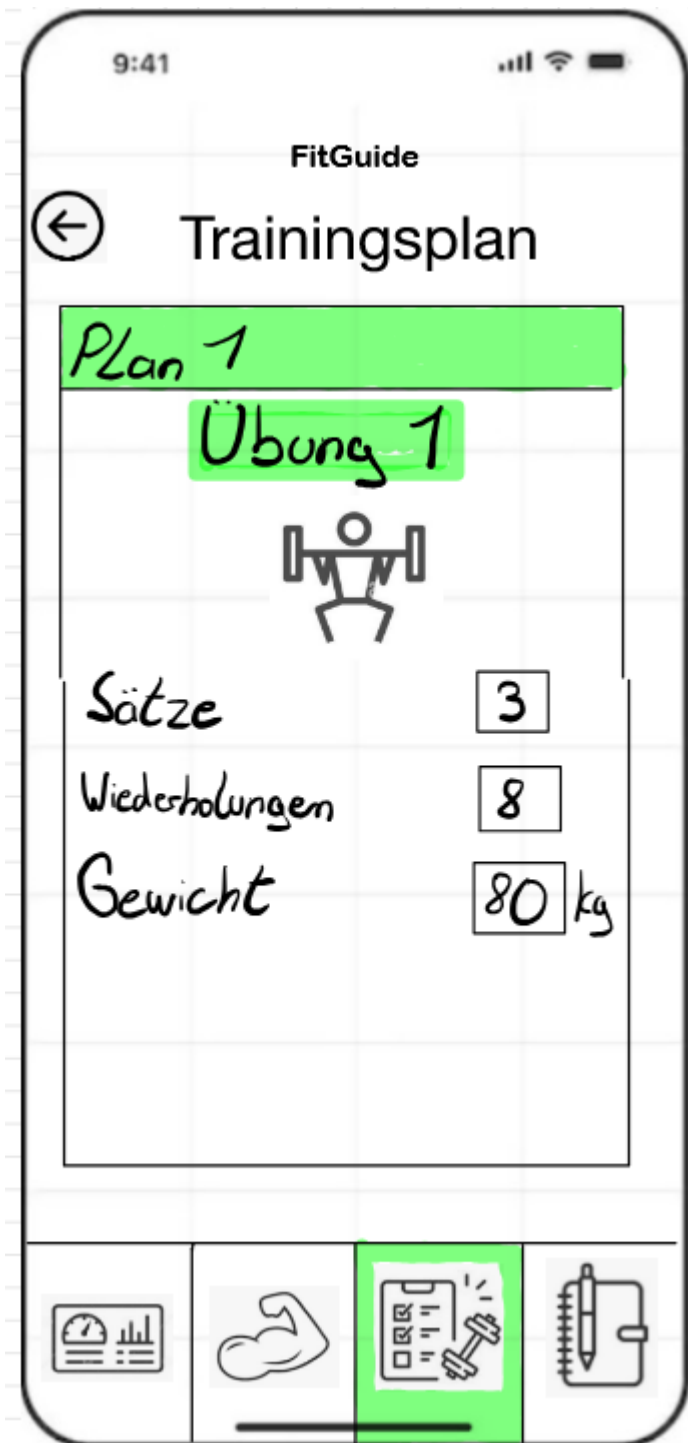
Übersicht aller erstellten Trainingspläne mit Workouts.





## Übung zu Trainingsplan hinzufügen

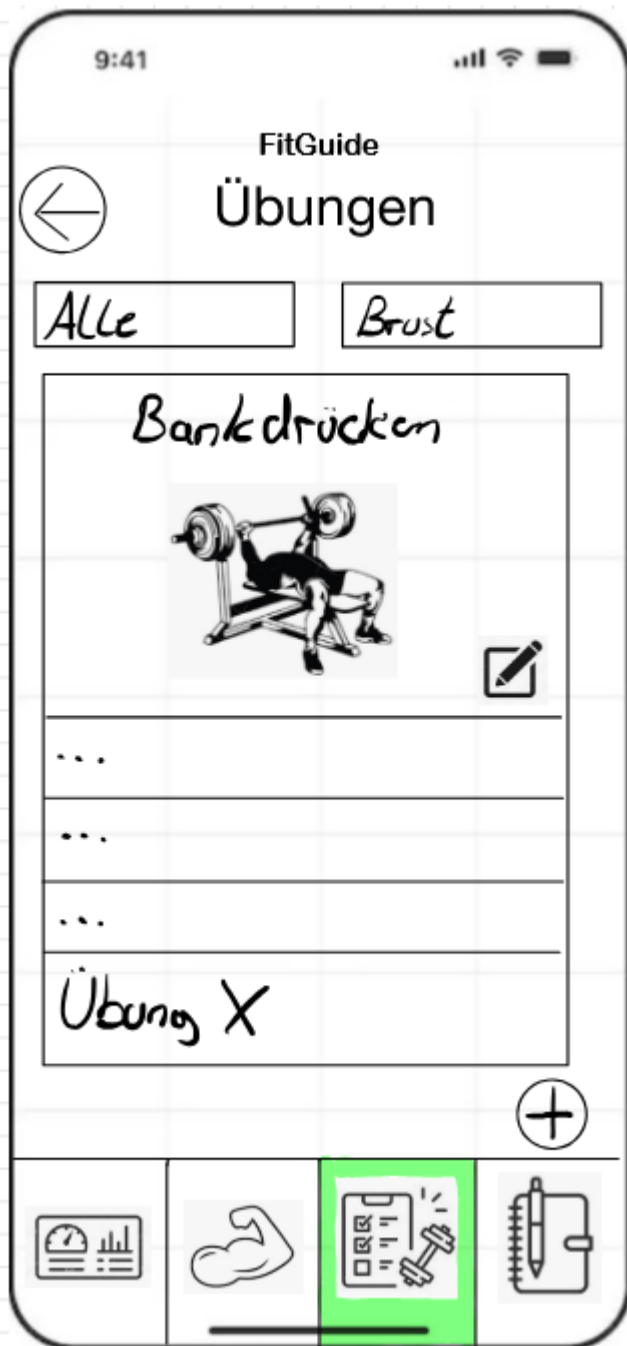
Auswahl und Hinzufügen von Übungen aus dem Pool zu einem Workout.



## Übungspool

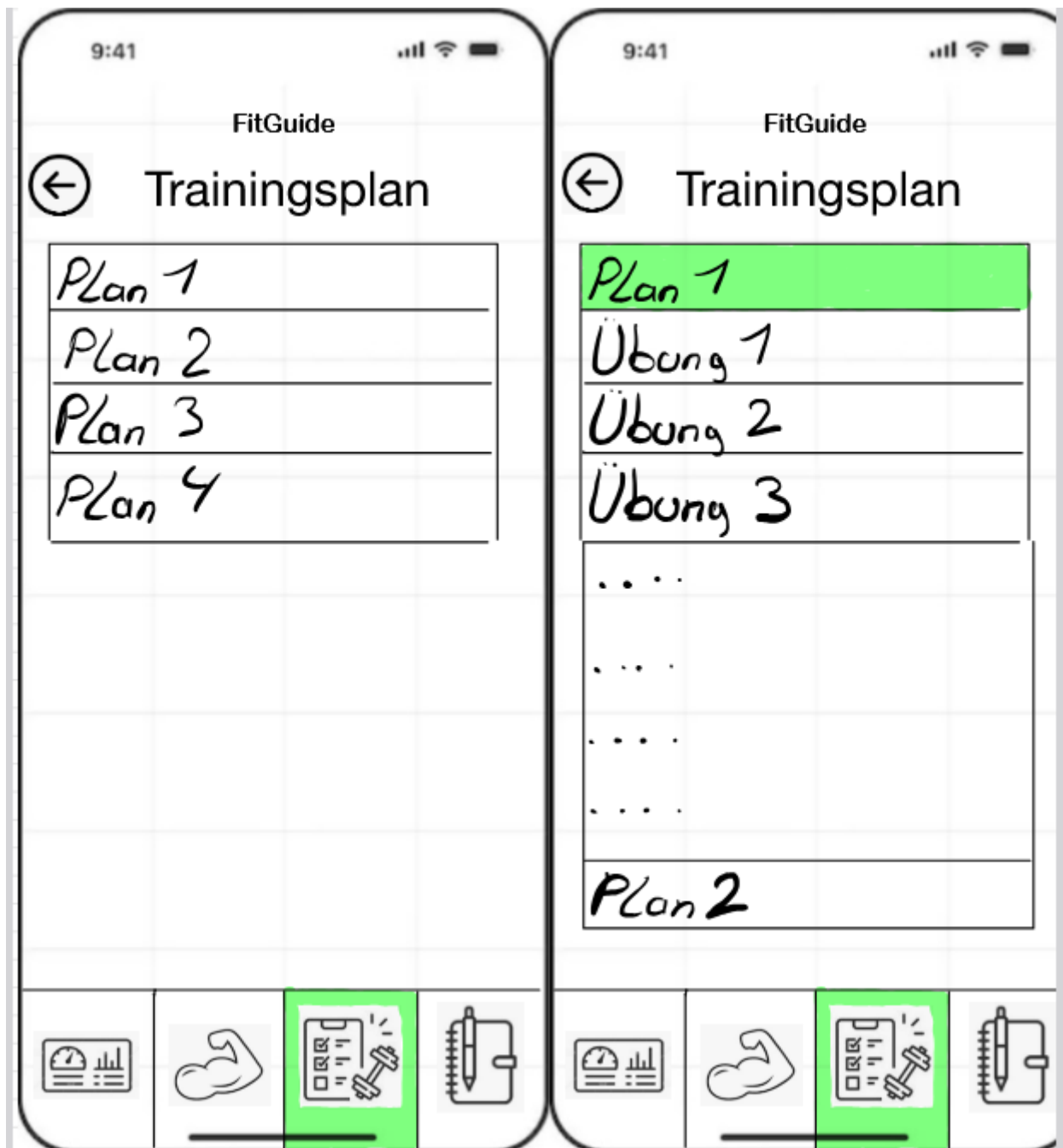
Verwaltung eigener Übungen mit Name und Muskelgruppe.





## Workout starten

Vorbereitung und Start eines Workouts aus dem Trainingsplan.



## Training durchführen

Live-Dokumentation während des Trainings mit Set-Logging.



4. Zeitplan

| Phase                       | Zeitraum                | Meilensteine                             |
|-----------------------------|-------------------------|------------------------------------------|
| Konzeption                  | 01.12.2024 - 12.12.2024 | Wireframes, User Stories, Projektplanung |
| Entwicklung Hauptfunktionen | 13.12.2024 - 20.01.2025 | Grundfunktionen, Core Features           |
| Feinschliff & Optimierung   | 20.01.2025 - 31.01.2025 | Bugfixing, UI-Verbesserungen, Testing    |
| Dokumentation               | 01.02.2025 - 05.02.2025 | Fertigstellung Dokumentation             |
| Abgabe                      | 06.02.2025              | Finale Abgabe                            |

5. Installationsanweisung / Kurzanleitung

## Voraussetzungen

- Flutter SDK: Version 3.24.5 oder höher
- Dart: Version 3.5.4 oder höher
- Android Studio / VS Code mit Flutter-Extensions
- Ein Android-Gerät oder Simulator

## Installation

### 1. Repository klonen:

```
git clone https://github.com/username/fitguide.git
cd fitguide
```

### 2. Dependencies installieren:

```
flutter pub get
```

### 3. App auf einem Gerät/Emulator starten:

```
flutter run
```

## Test-User-Account

Für Testzwecke kann folgender Account verwendet werden:

- **E-Mail:** christoph@web.de
- **Passwort:** 123456

Es wird empfohlen, sich selbst zu registrieren und die App auszuprobieren, um den kompletten Onboarding-Prozess zu durchlaufen.

## Schnelleinstieg

Nach der Registrierung und dem Onboarding erhält jeder neue Benutzer automatisch:

- **Übungspool** - Vorkonfigurierte Übungen für alle Muskelgruppen
- **Beispiel-Trainingspläne** - Trainingspläne basierend auf der angegebenen Trainingshäufigkeit
- **Makro-Empfehlungen** - Automatisch berechnete Kalorien- und Makronährstoff-Ziele

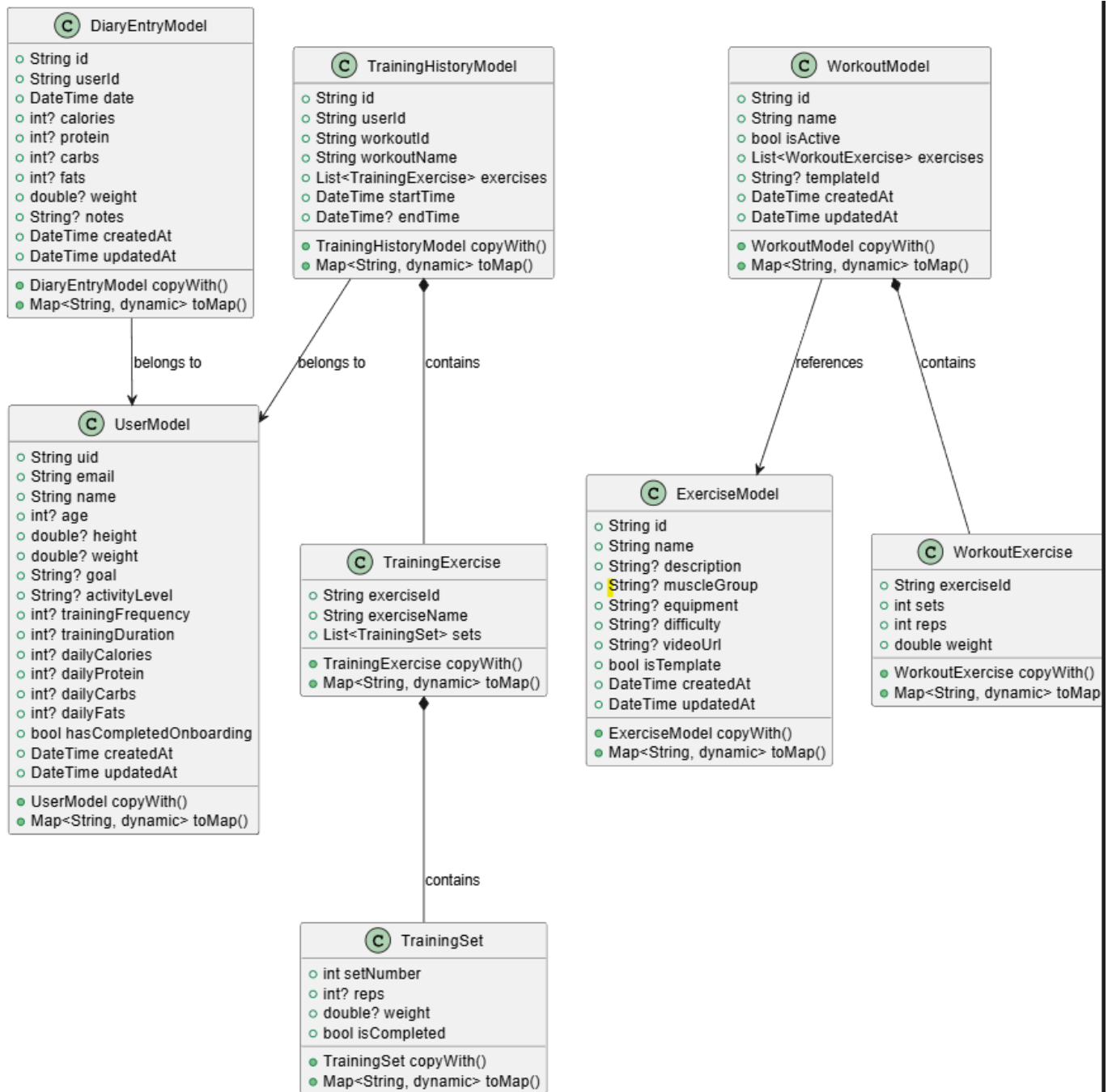
Die App ist sofort einsatzbereit und kann direkt genutzt werden!

---

## 6. Aufbau der Anwendung

### Klassendiagramm

Das folgende Diagramm zeigt die wichtigsten Modelle der Anwendung und deren Beziehungen zueinander. Es stellt die zentralen Datenstrukturen dar, die für die Kernfunktionalitäten (Benutzer, Training, Tagebuch) verwendet werden.



### Hauptmodelle:

- **UserModel**: Benutzerprofil mit Körperdaten und Makro-Zielen
- **DiaryEntryModel**: Tägliche Einträge mit Gewicht, Kalorien und Fotos
- **TrainingPlanModel**: Struktur der Trainingspläne mit Workouts
- **ExerciseModel**: Übungspool mit Muskelgruppen-Kategorisierung
- **TrainingHistoryModel**: Dokumentation abgeschlossener Trainings

### Anwendungsstruktur

Die Anwendung folgt einer klaren Hierarchie von der Initialisierung bis zur Benutzeroberfläche:



## 1. App-Start (main.dart)

- **Firestore-Initialisierung:** Beim App-Start wird Firestore initialisiert und konfiguriert
- **Global Providers:** AuthCubit und ThemeCubit werden als globale BlocProvider bereitgestellt
- **AuthWrapper:** Die App startet mit dem AuthWrapper, der als Einstiegspunkt dient

## 2. Authentication Flow (AuthWrapper)

Der AuthWrapper entscheidet basierend auf dem AuthState, welcher Screen angezeigt wird:

```
AuthState
├─ Unauthenticated → LoginScreen
├─ Authenticated (ohne Onboarding) → OnboardingScreen
└─ Authenticated (mit Onboarding) → MainScreen
```

**AuthCubit** überwacht den Firebase Authentication State und lädt automatisch Benutzerdaten aus Firestore. Bei Änderungen wird der entsprechende State emittiert.

## 3. Hauptnavigation (MainScreen)

Nach erfolgreicher Anmeldung und Onboarding gelangt der Benutzer zum MainScreen mit BottomNavigationBar:

- **Dashboard** - Übersicht mit Streak, Gewicht und Makros
- **Logbook** - Tagebuch für tägliche Einträge und Historie
- **Trainingsplanung** - Trainingsplanung
- **Settings** - Einstellungen und Logout

## 4. Feature-Module

Jedes Feature ist in einem eigenen Modul organisiert:

```
features/
├─ auth/           # Authentifizierung
├─ onboarding/     # Personalisierung
├─ dashboard/      # Hauptübersicht
├─ logbook/
│   └─ diary/      # Tagebuch-Einträge
│       └─ training_history/ # Trainingshistorie
└─ training_planner/
    ├── plan_builder/ # Trainingsplanerstellung
    ├── exercise_pool/ # Übungsverwaltung
    └─ workout_session/ # Trainingsausführung
```

Jedes Feature enthält:

- **screens/** - UI-Komponenten

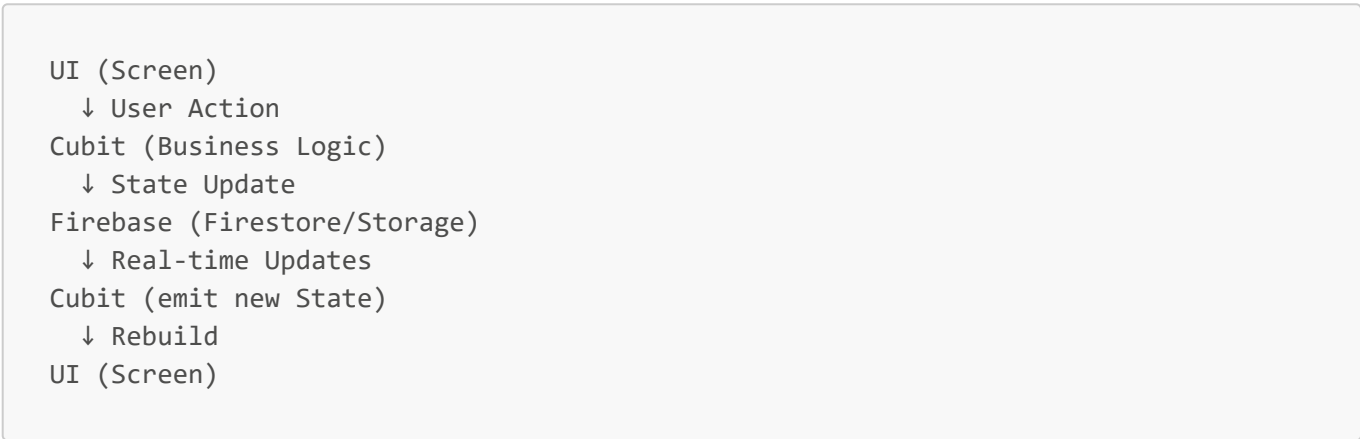
- **cubit/** (optional) - State Management (Cubit + State)
- **models/** (optional) - Datenmodelle
- **widgets/** (optional) - Wiederverwendbare Komponenten

5. Core Services

Zentrale Services im **core/** Ordner:

- **AuthService:** Firebase Authentication Wrapper
- **TemplateService:** Erstellung von Standard-Trainingsplänen beim Onboarding
- **ThemeCubit:** Verwaltung von Dark/Light Mode

6. Datenfluss



Die App nutzt StreamSubscriptions für Echtzeit-Updates aus Firestore, sodass Änderungen sofort in der UI reflektiert werden.

7. Angewendete Methoden und Bibliotheken

Entwicklungsmethoden

Architektur

- **Pattern:** Feature-First Architecture mit BLoC-Pattern
- **Ordnerstruktur:**
  - **lib/features/** - Funktionale Module (auth, dashboard, training, etc.)
  - **lib/core/** - Gemeinsame Services, Themes, Utilities
  - Jedes Feature enthält: screens, cubits, models, states

| Bibliothek       | Version | Zweck                      |
|------------------|---------|----------------------------|
| flutter_bloc     | ^8.1.6  | State Management mit Cubit |
| firebase_auth    | ^5.3.3  | Benutzer-Authentifizierung |
| cloud_firestore  | ^5.5.0  | Cloud-Datenbank            |
| firebase_storage | ^12.3.8 | Bild-Speicherung           |

| Bibliothek   | Version | Zweck                 |
|--------------|---------|-----------------------|
| fl_chart     | ^0.69.2 | Diagramme und Charts  |
| image_picker | ^1.1.2  | Foto-Upload vom Gerät |
| intl         | ^0.19.0 | Datumsformatierung    |
| equatable    | ^2.0.7  | State-Vergleiche      |

---

## 8. Technische Punkte

### Verwendete Technologien

- **Framework:** Flutter 3.24.5
- **Programmiersprache:** Dart 3.5.4
- **Datenbank:** Firebase Firestore
- **Backend:** Firebase (Authentication, Cloud Storage)
- **State Management:** flutter\_bloc (Cubit Pattern)

### Entwicklungsumgebung

- **IDE:** VS Code mit Flutter/Dart Extensions
  - **Versionskontrolle:** Git
  - **Testing:** flutter\_test für Unit Tests
  - **Firebase Console:** Backend-Verwaltung und Monitoring
- 

## 9. Tests

### Testkonzept

Die App verwendet flutter\_test für automatisierte Unit Tests. Der Fokus liegt auf der Testbarkeit von Business-Logik und Berechnungen. Komplexe Algorithmen wie die Makronährstoff-Berechnung werden isoliert getestet, um die Korrektheit zu gewährleisten.

### Unit Tests

#### Test 1: Kalorien-Berechnung für Muskelaufbau

- **Datei:** `test/features/onboarding/cubit/onboarding_cubit_test.dart`
- **Beschreibung:** Testet die Berechnung der täglichen Kalorienzufuhr basierend auf Körperdaten, Aktivitätslevel und Trainingsziel
- **Testfall:** 25-jähriger Mann, 80kg, 180cm, Moderately Active, Ziel: Muskelaufbau
- **Erwartetes Ergebnis:** 3098 Kalorien
- **Status:** Bestanden

### Manuelle Tests

Folgende Funktionen wurden manuell getestet:

- **Registrierung & Login:** E-Mail-Validierung, Passwort-Bestätigung, Fehlerbehandlung
- **Onboarding:** 5-Schritte-Prozess, Validierung aller Eingaben, Makro-Berechnung
- **Dashboard:** Streak-Berechnung, Gewichtsdiagramm mit Daten, Makro-Anzeige
- **Tagebuch:** Datumswahl, Dateneingabe, Foto-Upload, Persistierung
- **Trainingsplaner:** Plan erstellen/bearbeiten/löschen, Workout hinzufügen, Übungen zuweisen
- **Training:** Workout starten, Sets loggen, Training beenden, Historie speichern
- **Theme:** Dark/Light Mode wechseln, Persistierung

## Test-Ausführung

Tests können über die Kommandozeile ausgeführt werden:

```
# Alle Tests ausführen
flutter test

# Spezifischen Test ausführen
flutter test test/features/onboarding/cubit/onboarding_cubit_test.dart
```

---

## 10. Aktueller Stand

Alle geplanten User Stories für die erste Version der App wurden erfolgreich umgesetzt. Die Kernfunktionalität ist vollständig implementiert und getestet. Nutzer können sich registrieren, ein personalisiertes Onboarding durchlaufen, ihre Trainingspläne verwalten und sowohl Trainingseinheiten als auch Ernährung im Tagebuch dokumentieren.

Die App bietet eine solide Grundlage für einen erfolgreichen Einstieg in ein gesünderes Leben. Alle essentiellen Features, von der Workout-Planung über die Trainingsausführung bis hin zur Ernährungsdokumentation, funktionieren zuverlässig und intuitiv.

### Ausblick und geplante Erweiterungen

Für zukünftige Versionen sind mehrere intelligente Features geplant, die die App noch leistungsfähiger machen. Ein zentrales Ziel ist die automatische Anpassung der Makronährstoffempfehlungen basierend auf dem individuellen Fortschritt des Nutzers. Sobald das Gewicht erfasst wird und sich in Relation zum gesetzten Ziel entwickelt, soll die App die täglichen Kalorien- und Makroziele dynamisch anpassen. Wenn beispielsweise das Gewicht stagniert oder sich schneller als geplant verändert, wird die Ernährungsempfehlung entsprechend optimiert.

Ein weiteres wichtiges Feature ist das intelligente Performance-Tracking im Training. Die App soll nach jeder Trainingseinheit automatisch analysieren, wo Steigerungspotenzial besteht, und konkrete Empfehlungen aussprechen – etwa mehr Gewicht, zusätzliche Wiederholungen oder Sätze. Dadurch wird sichergestellt, dass Nutzer kontinuierlich Fortschritte machen und nicht in einer Trainingsroutine stagnieren.

Zudem ist eine Integration mit gängigen Tracking-Apps wie MyFitnessPal oder Apple Health geplant. Diese Schnittstellen würden es ermöglichen, Kaloriendaten und Makronährstoffe automatisch ins Tagebuch zu übernehmen, wodurch die manuelle Eingabe entfällt und die Nutzererfahrung deutlich vereinfacht wird.

Diese Erweiterungen würden die App von einem reinen Planungs- und Tracking-Tool zu einem intelligenten Coaching-Assistenten weiterentwickeln, der aktiv dabei unterstützt, Ziele zu erreichen und kontinuierlich besser zu werden.